Module 1 Formula Handbook — Lessons 01–05

Physical AI Core Curriculum | Module 1: Robotics Foundations

This handbook organizes the formulas used in Lessons 1–5. It is a study guide, not proof that every advanced robotics convention has been covered. Worked examples and learning boundaries reflect the completed lessons.

Quick-reference formula sheet

Lesson	Topic	Formula / rule	Conditions
01	Bounded gripper move	step = min(max_step, target - current)	Target ahead; positive direction only
01	State update	next_position = current_position + step	Simple 1D simulation
02	Relative displacement	d = p_object_world - p_robot_world	Aligned axes
02	3D position	p = (x, y, z)	Axes and origin must be defined
03	Signed angle change	delta_theta = theta_final - theta_initial	Same angle convention, unwrapped step
03	New orientation	theta_new = theta_old + delta_theta	Rotation around defined axis
04	2D pose	pose = (x, y, theta)	Defined planar frame
04	Learning 3D pose	pose = (x,y,z,roll,pitch,yaw)	Euler convention/order must be stated
05	2D rotation matrix	Rot(theta) = [[cos(theta),-sin(theta)],[sin(theta),cos(theta)]]	Positive is counterclockwise
05	Inverse rotation	Rot(-theta) = transpose(Rot(theta))	Pure 2D rotation
05	World to robot	p_R = Rot(-theta) (p_W - t_W)	Rigid planar frame
05	Robot to world	p_W = Rot(theta) p_R + t_W	Rigid planar frame
05	Parent-child	p_W = Rot(theta_parent) p_local + t_parent	XY planar or 3D yaw-only example

Notation used throughout

• `p_W`: point or object position expressed in world coordinates.

• `p_R` or `p_local`: the same position expressed in the robot/parent's local frame.

• `t_W`: robot/parent frame origin expressed in world coordinates.

• `theta`: robot yaw measured counterclockwise relative to world X in the XY plane.

• `delta_theta`: signed change in orientation.

• `Rot(theta)`: active counterclockwise rotation; use the inverse to change from world to rotated robot-frame coordinates.

• X, Y, Z are coordinates, not automatically motion commands or world directions unless the frame is defined.

Lesson 01 — Physical AI: Feedback and action updates

Core idea: sense -> decide -> act -> observe again. A policy maps observations to actions; the environment changes after each action.

For the guided 1D gripper case, the cup is ahead of the gripper and maximum safe step is 2 cm:

```
distance = cup_position - gripper_position
step = min(2, distance) # only if distance > 0
gripper_position = gripper_position + step
```

Example: gripper 8 cm, cup 9 cm -> remaining 1 cm -> step 1 cm, not 2 cm. The rule avoids overshooting in this simplified forward-only scenario.

Important limitation: This is not a general robot controller; it does not handle obstacles, reverse movement, dynamics, sensor noise, or collisions.

Lesson 02 — 2D / 3D coordinates and aligned frames

2D position: `(x,y)`. 3D position: `(x,y,z)`.

For frames with the same axis directions:

```
relative_point = world_point - frame_origin_world
```

Example: robot world (3,2), cup world (8,5) -> robot relative cup (5,3).

Caution: Plain subtraction alone is insufficient if frame axes are rotated.

Lesson 03 — Orientation and signed angular changes

Position describes *where* an object is. Orientation describes *which direction* it faces.

```
delta_theta = theta_final - theta_initial
theta_final = theta_initial + delta_theta
```

Example: 45 degrees -> 90 degrees: signed change +45 degrees. From 90 degrees -> 0 degrees: signed change -90 degrees. Clockwise is negative in the agreed top-down convention.

Roll, pitch, yaw correspond to X, Y, Z rotations in the examples; actual interpretation depends on axis definition and Euler order. Angle wraps mean equivalent orientations can have different numerical angles (e.g. +180 and -180 degrees).

Lesson 04 — Pose combines position and orientation

Planar robot pose:

```
pose_2d = (x, y, theta)
```

Example: `(2,3,0 degrees)` moved +2 world-X, +1 world-Y and yaw +90 degrees -> `(4,4,90 degrees)`.

Introductory Euler 3D pose:

```
pose_3d = (x, y, z, roll, pitch, yaw)
```

Example Isaac Sim Checkpoint 1: `(2,1,1.5,0,0,45 degrees)`; later pose `(4,1,1.5,0,0,90 degrees)`; change in X +2 and yaw +45 degrees.

Pose changes when position OR orientation changes. Euler angles are useful for learning but require explicit conventions and may be singular in some configurations.

Lesson 05 — Translation, rotation and frame transforms

Define the 2D rotation matrix:

```
Rot(theta) = [ [ cos(theta), -sin(theta) ],
[ sin(theta), cos(theta) ] ]
```

For theta = +90 degrees:

```
Rot(+90) = [[0,-1],[1,0]]
Rot(-90) = [[0,1],[-1,0]]
```

World to robot: subtract the frame origin, then inverse-rotate.

```
p_R = Rot(-theta) * (p_W - t_W)
```

Worked example: frame origin `(3,2)`, yaw `+90 degrees`, world point `(7,5)`.

• Subtract translation: `(7,5) - (3,2) = (4,3)`.

• Apply Rot(-90): `(4,3) -> (3,-4)`.

• Robot coordinate: `(3,-4)`.

Robot to world: rotate the local vector, then add the frame origin.

```
p_W = Rot(theta) * p_R + t_W
```

Worked example: frame origin `(2,3)`, yaw `+90 degrees`, local point `(5,-2)`.

• Rotate with Rot(+90): `(5,-2) -> (2,5)`.

• Add translation: `(2,5) + (2,3) = (4,8)`.

• World coordinate: `(4,8)`.

Isaac Sim parent-child example: parent origin `(2,3,0)`, child local `(1,0,1)`.

• Parent yaw 0 degrees -> child world `(3,3,1)`.

• Parent yaw +90 degrees -> child world `(2,4,1)`.

• Child local Translate stays `(1,0,1)` in both cases.

Five mistakes to catch during review

1. Mixing up object movement with changes in numerical coordinates due to different reference frames.
2. Using simple subtraction for rotated frames without the inverse rotation.
3. Rotating first and then subtracting for world-to-robot (wrong order).
4. Mistaking a signed angular change for the absolute orientation.
5. Assuming an Isaac Sim USD file records prior intermediate editing states.

Review drill — Try without looking at solutions

1. A gripper is at 8 cm, cup at 9 cm, max step 2 cm. What is the safe forward move?
2. Robot at (2,1), cup world (6,4), axes aligned. Cup in robot frame?
3. Angle changes from 90 degrees to 0 degrees. Signed delta?
4. Is pose changed if a robot turns in place?
5. A robot at world (3,2) yaw +90 degrees observes world point (7,5). Point in robot frame?
6. Robot origin (2,3), yaw +90 degrees, child point local (1,0). Child XY world?
7. Which calculation goes first in robot-to-world: translation or rotation?

Review answers

1. +1 cm.
2. (4,3).
3. -90 degrees.
4. Yes, orientation changes.
5. (3,-4).
6. (2,4).
7. Rotate first, then add translation.

Scope / next milestone

Module 1, Lessons 1–5 are completed and documented according to the chat. Lesson 5 USD save and prior USD saves are user-confirmed; the binary scene files themselves have not been uploaded. Lesson 6 has not started. Coordinate transformations here are 2D or simplified yaw-only 3D; full 3D rotation matrices, quaternions and homogeneous transforms are future study.